L’objectif de ce notebook est de **démontrer la faisabilité d’une classification supervisée ultérieure** en vérifiant que les images de produits peuvent être séparées visuellement selon leurs catégories principales.
Pour cela, nous utilisons une approche en deux temps :

1. **Extraction de caractéristiques visuelles** via un réseau CNN pré-entraîné (VGG16)
2. **Analyse non supervisée** des représentations obtenues par réduction de dimension et clustering

Si les catégories sont naturellement séparées dans l’espace des features, une classification supervisée aura de fortes chances de bien fonctionner.

Images produits > VGG16 (pré-entraîné) > Vecteurs de features (4096D) > PCA (réduction dimensionnelle) >

T-SNE (projection 2D) > KMeans (clusters théoriques) > ARI + Matrice de confusion > Validation de faisabilité

Étape 0 — Chargement et structuration des données

. data : DataFrame principal
. image_path : chemin vers le fichier image
. label_name : catégorie principale textuelle (ex: "kitchen & Dining)
. label : Encodage numérique de "label_name"
. def photo_class : Fonction qui Associe chaque image à une catégorie principale en analysant son chemin ou son nom de fichier.

Étape 1 — Création du modèle CNN pré-entraîné

. model : Réseau CNN tronqué qui transforme une image en un vecteur de features numériques
. base_model = VGG16() : On utilise **VGG16 pré-entraîné sur ImageNet** comme extracteur de features.
. layers[-2] : . La dernière couche est une couche de classification
. L’avant-dernière couche contient un **vecteur de caractéristiques visuelles de haut niveau**
. Ce vecteur devient notre "empreinte visuelle" pour chaque image

Étape 2 — Extraction des features des images

Boucle principale:
. images_features : liste des vecteurs CNN extraits
. valid_idx : indices des images traitées avec succès

Traitement par image:
. load_img : Redimensionne l'image . img_to_array : convertit en tenseur . expand_dims: Ajoute dimension batch
. preprocess_input: normalise selon VGG16 . predict : extrait les features CNN

. shape = (N,4096) : N = nombre d'images valides 4096 = taille du vecteur VGG16

Alignement des données
Certaines images peuvent échouer au chargement. On réaligne donc les labels pour garantir que chaque vecteur de features correspond bien à la bonne catégorie:

```
data_valid = data.iloc[valid_idx].reset_index(drop=True)
```

Étape 3 — Réduction de dimension PCA

Objectif
Réduire la dimension tout en conservant 99% de la variance.

. feat_pca : Représentation compacte des images
. Dimension : ~ 180 - 300 colonnes au lieu de 4096

Étape 4 — Projection T-SNE

Objectif
Projeter les images dans un espace 2D pour :

. Visualiser la structure des données
. Observer la séparabilité des catégories

X_tsne : coordonnées 2D des images
. tsne1: axe X . tsne2: axe Y . class: catégorie réelle

Étape 5 — Clustering KMeans

Objectif
Créer des clusters "théoriques" pour mesurer leur correspondance avec les vraies classes

. cluster: Groupe assigné à chaque image

Étape 6 — Mesure ARI

L'**Adjusted Rand Index (ARI)** mesure la similarité entre : . Les vraies classes
. Les clusters générés

Interprétation Score:
≈ 0 (aléatoire) > 0.2 (strucure existante) > 0.4 Forte correspondance

Étape 7 — Matrice de confusion

But :
Comprendre quelles catégories se confondent visuellement

Transformation cluster → classe
. conf_mat_transform() : Les clusters n'ont pas d'ordre sémantique, donc on les remappe vers les vraies classes

réusltat:
. Matrice finale classe réelle vs classe prédite
. Rapport de clustering lisible comme une classification

Étape 8 — Analyse d'images mal classées

But
Visualiser les images responsables des confusions
. list_misclassified

Ce que démontre ce pipeline
Les catégories principales sont globalement séparables dans l'espace des features extraits par VGG16

Implication métier
- Les images contiennent une structure visuelle exploitable
- Une **classification supervisée CNN fine-tunée est pertinente et prometteuse**

Images produits > VGG16 (pré-entraîné) > Vecteurs de features (4096D) > PCA (réduction dimensionnelle) >

T-SNE (projection 2D) > KMeans (clusters théoriques) > ARI + Matrice de confusion > Validation de faisabilité

Ce notebook démontre que les représentations visuelles extraites par un réseau de neurones convolutif pré-entraîné structurent naturellement les images produits selon leurs catégories métiers. Malgré la complexité visuelle et la diversité des produits, les clusters obtenus présentent une forte cohérence avec les classes réelles, validant ainsi la faisabilité d'une approche supervisée pour la classification automatique du catalogue.

Comparaison finale des approches

1_ Approche simple (baseline) — Transfer Learning VGG16

Résultats principaux :
- Test Accuracy ≈ **0.79**
- Loss ≈ **0.95**
- Courbes :
  - Train accuracy → très haute (~0.98)
  - Val accuracy → ~0.85
- Matrice de confusion :
  - Bonnes prédictions globales
  - Confusions sur classes proches (ex : Home Decor vs Furnishing)

Interprétation :
- Modèle apprend très bien
- Mais **overfitting léger** :
  - Train >> Validation
- Normal sans augmentation

Sert de baseline de référence

2_ ImageDataGenerator + Data Augmentation

Résultats :
- Validation Accuracy ≈ **0.77**
- Test Accuracy ≈ **0.76**

Observation :
- Accuracy plus basse que baseline
- Loss plus élevée

Pourquoi ?
Parce que l'augmentation :
- Rend la tâche plus difficile
- Ajoute du bruit contrôlé
- Force la généralisation

Le modèle "triche moins" → il mémorise moins
C'est normal d'avoir une accuracy brute plus faible.

3_ Dataset API (sans augmentation)

Résultats :
- Validation Accuracy ≈ **0.76**
- Test Accuracy ≈ **0.79**

Analyse :
- Très proche baseline
- Pipeline plus moderne
- Chargement plus optimisé GPU

Avantage : performance technique, pas accuracy.

4_ Dataset + Augmentation intégrée au modèle

Résultats :
- Training Accuracy ≈ **0.93**
- Validation Accuracy ≈ **0.83–0.85**
- Test Accuracy ≈ **0.83**

Meilleur compromis global.

Comparaison synthétique

Approche	Val Acc	Test Acc	Overfitting	Robustesse
Baseline simple	~0.85	~0.79	Moyen	Moyenne
ImageDataGenerator	~0.77	~0.76	Faible	Bonne
Dataset sans aug	~0.76	~0.79	Moyen	Moyenne
Dataset + aug modèle	**~0.84**	**~0.83**	Faible	**Excellente**

Lecture des courbes globales

Overfitting visible quand :
- Train accuracy monte
- Val accuracy stagne/baisse
- Train loss ↓
- Val loss ↑

Observé dans baseline.

Bon apprentissage quand :
- Train & Val montent ensemble
- Loss ↓ des deux côtés

Observé avec augmentation intégrée.

Impact GPU / temps d'entraînement
Constat :
- CPU → plusieurs heures
- GPU T4 → quelques minutes

L'utilisation du GPU a permis de réduire drastiquement le temps d'entraînement, rendant possible l'expérimentation de plusieurs pipelines.

Conclusion finale (prête pour rapport)

Conclusion comparative

Plusieurs approches de classification supervisée d'images ont été expérimentées à partir du modèle VGG16 en Transfer Learning.

L'approche baseline sans augmentation atteint une accuracy test d'environ 79 %, mais présente un surapprentissage notable, visible par l'écart entre performances d'entraînement et de validation.

L'utilisation de l'ImageDataGenerator avec data augmentation réduit ce surapprentissage mais au prix d'une baisse de l'accuracy brute, due à une plus grande difficulté d'apprentissage induite par les transformations d'images.

L'approche basée sur l'API Dataset de TensorFlow, plus récente, offre des performances similaires tout en optimisant le pipeline de chargement des données.

Enfin, l'intégration de la data augmentation directement dans le modèle via des couches dédiées fournit le meilleur compromis :

- Meilleure généralisation
- Réduction de l'overfitting
- Accuracy test supérieure (~83 %)

Cette approche constitue donc la solution retenue pour la classification supervisée des images produits.

Pourquoi l’accuracy baisse avec l’augmentation ?

Sans augmentation
Le modèle voit :
- Toujours les mêmes images
- Toujours les mêmes angles
- Toujours les mêmes cadrages

Donc il apprend :
Les détails spécifiques des images
- Le bruit
- Les artefacts

Il “mémorise” plus qu’il ne “comprend”
Résultat :
- Train accuracy très haute
- Test accuracy parfois bonne… mais fragile

Avec augmentation
On modifie les images :
- Rotation
- Flip
- Zoom
- Shift

Donc le modèle voit :
- Des variantes
- Des positions différentes
- Des objets déformés

Il doit apprendre :
- Les vraies caractéristiques
- Pas les pixels exacts

Résultat :
- Apprentissage plus difficile
- Accuracy train plus basse
- Généralisation meilleure

Sans augmentation : “L’élève apprend les réponses par cœur.” → meilleur score à l’entraînement

Avec augmentation : “L’élève apprend à comprendre les questions.” → meilleur score en situation réelle

Dataset trop petit (~1050 images.)

Data augmentation :
- Augmente la difficulté
- Mais pas forcément la performance immédiate

Sur petit dataset :
Le gain n’est pas toujours visible

Augmentation trop forte peut :
- Déformer trop les objets
- Rendre certaines classes ambiguës

L’approche simple obtient une accuracy test légèrement supérieure (~0.80) à l’approche avec data augmentation (~0.76).
Cependant, les courbes d’apprentissage montrent un surapprentissage marqué pour le modèle simple, avec un écart important entre accuracy d’entraînement et de validation.
L’augmentation de données réduit cet écart et améliore la capacité de généralisation du modèle, au prix d’une accuracy brute légèrement inférieure.

Pourquoi l’augmentation fait moins bien ?

Parce qu’elle rend l’apprentissage plus difficile mais plus réaliste. Le modèle simple surapprend, alors que l’augmentation améliore la généralisation, ce qui est préférable en conditions réelles.

EDA TEXTE (Exploratory Data Analysis)

Objectif
Analyser les descriptions textuelles des produits e-commerce afin de :
- Comprendre le contenu des données
- Identifier les mots dominants par catégorie
- Vérifier si le texte permet de distinguer les classes
- Préparer une future modélisation NLP

Dataset utilisé :
- Descriptions produits Flipkart
- Catégories : Baby Care, Computers, Watches, etc.

2.1 Passage en minuscules
text = text.lower()

- Uniformiser les mots
- “Watch” = “watch”

2.2 Suppression ponctuation / caractères spéciaux
Fonctions utilisées :
import re

text = re.sub(r"[^a-zA-Z]", " ", text)

- Supprime chiffres, symboles
- Garde uniquement lettres

2.3 Tokenisation
Découper le texte en mots :
tokens = text.split()

Exemple :
"blue cotton baby shirt"
→ ["blue","cotton","baby","shirt"]

2.4 Suppression des stopwords
Stopwords = mots inutiles :
- the - and - for - with

Librairie :
from nltk.corpus import stopwords

Code :
stop_words = set(stopwords.words("english"))

tokens = [w for w in tokens if w not in stop_words]

2.5 Lemmatisation
Ramener les mots à leur racine :
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

tokens = [lemmatizer.lemmatize(w) for w in tokens]

Exemples :

Avant	**Après**
watches	watch
babies	baby

3- Fonction de nettoyage globale
def clean_text(text):

Vectorisation du texte
Transformer le texte en nombres.
Deux approches classiques utilisées

4.1 Bag of Words (CountVectorizer)
Principe : Compter combien de fois chaque mot apparaît

4.2 TF-IDF
Pondère les mots rares
Diminue l’importance des mots fréquents

5- Analyse de fréquence des mots
- Identifier les mots dominants

Exemple :
word_freq = X_counts.sum(axis=0)
Ou via dictionnaire :
freq_dict = dict(zip(vectorizer.get_feature_names_out(), word_freq.tolist()[0]))

6- Visualisation — WordCloud
Librairie : from wordcloud import WordCloud

Analyse par classe
Voir si chaque catégorie a son vocabulaire.
Exemple :
df_baby = df[df["main_class"] == "Baby Care"]

Une analyse exploratoire des descriptions produits a été réalisée afin d’évaluer la pertinence de l’information textuelle pour la classification.

Après nettoyage (minuscule, suppression ponctuation, stopwords, lemmatisation), les textes ont été vectorisés via Bag-of-Words et TF-IDF. L’analyse de fréquence et les WordClouds ont mis en évidence des vocabulaires spécifiques à chaque catégorie, confirmant le potentiel discriminant du texte.

Toutefois, certaines classes présentent un chevauchement lexical, justifiant l’exploration complémentaire de la classification par l’image.

« La machine qui apprend à reconnaître les produits »

Chapitre 1 — Le grand carton d'images
Un jour, on t'a donné un grand carton rempli de produits e-commerce 🛍️ :
- Des montres
- Des ordinateurs
- Des coussins
- Des ustensiles de cuisine
- Des produits de beauté…

Mais tout était mélangé

Dans le carton, il y avait deux choses :
1. **Un fichier Excel (CSV)**
   → avec les infos des produits
2. **Un dossier d'images**
   → les photos de chaque produit

Mon objectif :
Construire une machine capable de reconnaître automatiquement la catégorie d'un produit à partir de son image.

Chapitre 2 — Observer avant d'agir (EDA)
Avant d'apprendre quoi que ce soit à la machine, on fait une analyse exploratoire

J'ai ouvert le fichier CSV et regardé :
- Combien il y a de produits
- Combien de catégories
- S'il manque des images
- Si certaines catégories sont trop petites

J'ai découvert :
- 7 grandes catégories principales
- Des déséquilibres entre classes
- Des noms de catégories parfois complexes

Chapitre 3 — Peut-on vraiment reconnaître les produits ?
Avant de construire le modèle final, je me suis posé une question :

« Est-ce que les images sont reconnaissables automatiquement ? »
Alors j'ai utilisé une machine adapté pour cela :
VGG16
C'est un cerveau artificiel entraîné sur 1,4 million d'images.
Il sait reconnaître :
- Chiens
- Voitures
- Meubles
- Objets…

je ne lui ai pas demandé de classer, mais juste :
« Décris-moi chaque image avec des chiffres. »
Ces chiffres s'appellent des **features**.

Chapitre 4 — Réduire la complexité
Le problème :
Chaque image devenait des milliers de chiffres 😱
Trop compliqué à visualiser.
Alors j'ai utilisé deux outils primordials :
PCA
Réduit les dimensions sans perdre l'essentiel.
t-SNE
Projette les images en 2D pour les voir.
Résultat :
j'ai vu des groupes se former.
Les montres ensemble.
Les ordinateurs ensemble.
Les coussins ensemble.
Bonne nouvelle : les images sont séparables !

Chapitre 5 — Le clustering
Ensuite j'ai testé :

« Si je laisse la machine regrouper toute seule, est-ce qu'elle retrouve les vraies catégories ? »
Tu as utilisé :
- **KMeans**

Il a créé des groupes.
Puis on a comparé avec les vraies classes grâce au score :
- **ARI (Adjusted Rand Index)**

Résultat :
Les clusters correspondaient bien aux catégories.
Conclusion faisabilité :
Oui, la classification supervisée est pertinente.

Chapitre 6 — Construire la vraie machine
Maintenant, tu passes au vrai objectif :
Apprendre à la machine à classer les images.
on reprends VGG16, mais avec quelques modification.
C'est le principe du :
Transfer Learning
On gardes son "cerveau visuel"
Et on ajoutes :
- Dense layers
- Dropout
- Softmax final (7 classes)

Chapitre 7 — Les 4 entraînements
Tu as testé 4 façons d’apprendre.

Approche 1 — Préparation simple
Tu charges toutes les images.
Tu les prétraites.
Tu entraînes.
Résultat :
- Bonne accuracy
- Pipeline simple
- Mais lourd en mémoire

Approche 2 — ImageDataGenerator
Tu ajoutes de la **data augmentation** :
- Rotation
- Flip
- Décalage

But :
Simuler plus d’images.
Résultat :
- Modèle plus robuste
- Mais parfois accuracy test légèrement plus faible

Pourquoi ?
Parce qu’il généralise plus, il sur-apprend moins.

Approche 3 — Dataset TensorFlow
Nouvelle méthode moderne :
- Plus rapide
- Optimisée GPU
- Pipeline propre

Résultat :
- Très bon compromis
- Stable

Approche 4 — Dataset + Augmentation intégrée
L’approche la plus moderne :
Augmentation directement dans le modèle :
- RandomFlip
- Rotation
- Zoom

Avantages :
- GPU optimisé
- Pipeline clean
- Meilleure généralisation

Chapitre 8 — La puissance du GPU
Au début :
3 heures d’entraînement CPU
Puis tu es passé sur Google Colab GPU :
Quelques minutes
Cela m'a permis d'apprendre :
- Monter Drive
- Sauvegarder modèles
- Recharger sans réentraîner

Chapitre 9 — Lire les résultats
J'ai analysé :
- Accuracy train
- Accuracy validation
- Accuracy test
- Loss

J'ai appris à lire :
- Overfitting
- Underfitting
- Early stopping
- Learning curves

🏁 **Chapitre 10 — La conclusion finale**
on a comparé les 4 approches.
j'ai constaté :
- Toutes fonctionnent bien
- Transfer Learning très efficace
- Data augmentation améliore la robustesse
- Dataset TF plus moderne
- GPU indispensable en production

Accuracy test ≈ 0.79 – 0.82 selon approches
Très bon score vu la taille du dataset.

Morale de l’histoire
Tu es parti de :
Un carton d’images mélangées
Tu as :
1. Exploré les données
2. Vérifié la faisabilité
3. Testé le clustering
4. Construit un modèle supervisé
5. Comparé plusieurs pipelines
6. Optimisé avec GPU
7. Analysé les performances

Et tu as fini avec :
Une machine capable de reconnaître automatiquement les catégories produits.

. **Slides 6–7 — EDA**
- 7 catégories
- 150 produits chacune
- Longueur moyenne des descriptions
- Forte hétérogénéité

Point important :
Certaines catégories ont peu de texte (Watches).

Les descriptions étant très inégales, cela impactera la qualité du clustering texte.
Très bonne observation métier.

Slides 8–12 — Partie TEXTE
Tu testes plusieurs méthodes.
Bag of Words / TF-IDF
ARI faible (~0.22 max)
Word2Vec
Silhouette correct mais ARI faible
Glove / FastText
Toujours faible
BERT
ARI = 0.567 (bien supérieur)

Les modèles contextuels comme BERT capturent mieux la sémantique que les méthodes classiques.
Conclusion logique :
Si on veut clusteriser du texte → BERT est recommandé.
Très bonne progression pédagogique.

Slides 13–20 — Partie IMAGE classique (SIFT + BoVW)
Tu expliques :
1. Extraction SIFT
2. Clustering des descripteurs
3. Histogrammes
4. PCA
5. T-SNE
6. KMeans
7. ARI

Résultat :
ARI ≈ 0.08 → faible

Les méthodes classiques basées sur les textures sont limitées pour des objets complexes.
Bonne transition vers CNN.

Slides 21–25 — CNN Transfer Learning
Tu expliques :
- VGG16 pré-entraîné
- Extraction de vecteurs 4096
- PCA
- T-SNE
- KMeans

Résultat :
ARI ≈ 0.45
(contre 0.08 avec SIFT) :

Sans supervision, les images se regroupent naturellement par catégorie.
Conclusion logique :
La classification supervisée est prometteuse.
C'est une excellente démonstration de faisabilité.

Slides 26–31 — Classification supervisée
Tu testes 4 approches.

Approche simple
Accuracy test : 79.5%
Surapprentissage visible.

ImageDataGenerator
Accuracy test : 76.1%
Moins d'overfitting mais accuracy brute plus faible.
Normal, augmentation = plus difficile.

Dataset sans augmentation
Accuracy test : 79%
Pipeline plus moderne.

Dataset + augmentation intégrée
Accuracy test : 83.8%
Meilleur résultat.
Meilleure généralisation.

Slide 32 — Conclusion

- Baseline correcte mais overfitting
- Augmentation réduit surapprentissage
- Dataset TF optimise pipeline

Augmentation intégrée = meilleur compromis
- **J'ai d'abord analysé les données pour comprendre leur structure.**
- **J'ai testé des approches non supervisées sur texte et images pour vérifier la faisabilité.**
- **Les méthodes classiques étaient limitées, mais le CNN pré-entraîné a montré une forte capacité naturelle à regrouper les catégories.**
- **J'ai ensuite développé plusieurs modèles supervisés et comparé leurs performances.**
- **L'approche Dataset TensorFlow avec data augmentation intégrée offre le meilleur compromis avec environ 83% d'accuracy test.**

PARTIE TEXTE

Page 6 — Graphique distribution longueurs descriptions
Ce qu'on voit :
- Tableau des 7 catégories
- Longueur moyenne des descriptions
- Statistiques (moyenne ≈ 76 mots, médiane = 44)
- Forte asymétrie (longue traîne)

Comment l'expliquer :

Les descriptions sont très hétérogènes. Certaines catégories comme Watches ont peu de texte, d'autres beaucoup.
Cela rend le clustering texte plus difficile car l'information disponible varie fortement.

Page 7 — Graphique "Avant vs Après preprocessing"
Ce qu'on voit :
Barres bleues = avant nettoyage
Barres orange = après nettoyage
Pour chaque catégorie, le nombre moyen de mots diminue légèrement.
Interprétation :

Le preprocessing supprime les mots inutiles (stopwords, ponctuation).
La structure générale reste la même mais le bruit est réduit.

Page 8 — PCA Bag-of-Words / TF-IDF
Ce qu'on voit :
Deux scatter plots (PC1 vs PC2)
- À gauche : Bag-of-Words
- À droite : TF-IDF

Chaque point = un produit
Couleur = cluster KMeans
ARI faible (~0.14 à 0.22)
Explication :

Les clusters sont très mélangés.
Les catégories ne se séparent pas clairement.
Conclusion : méthodes basiques peu performantes.

Page 9 — Word2Vec
Ce qu'on voit :
Projection PCA 2D
Les groupes semblent plus compacts visuellement.
Silhouette = 0.368
Mais ARI = 0.112 (faible correspondance réelle)
À dire :

Word2Vec crée une structure interne, mais elle ne correspond pas bien aux vraies catégories métier.

Page 10 — Glove / FastText
Ce qu'on voit :
Deux projections similaires, clusters encore mélangés.
ARI ≈ 0.14–0.19
Message :

Les embeddings classiques améliorent légèrement la structure, mais restent insuffisants.

Page 11 — BERT / USE
Ce qu'on voit :
Clusters bien plus distincts.
BERT → ARI = 0.567
USE → ARI = 0.264
À dire :

BERT capture le contexte des phrases, ce qui améliore fortement la séparation des catégories.
Visuellement, on voit des groupes plus nets.

🖼️ PARTIE IMAGE — Méthode classique

Page 14 — Exemples images par label
Ce qu'on voit :
Images de Home Decor, Computers, Watches
Pourquoi important :

Cela permet de vérifier visuellement que les catégories sont cohérentes et distinctes.

Page 15 — SIFT Keypoints
Ce qu'on voit :
Une montre avec des petits points verts (keypoints).
Nombre de keypoints = 1270
Descripteurs = (1270, 128)
Explication :

SIFT détecte des motifs locaux (textures, coins).
Mais il ne comprend pas le "sens" de l'objet.

Page 18 — T-SNE SIFT
Ce qu'on voit :
À gauche : vraies catégories
À droite : clusters KMeans
Beaucoup de mélange.
ARI = 0.081
Message :

Les catégories ne sont pas bien séparées avec SIFT.

Page 19 — Matrice de confusion SIFT
Ce qu'on voit :
Beaucoup de valeurs hors diagonale.
Interprétation :

Les clusters ne correspondent pas aux catégories réelles.

🧠 PARTIE IMAGE — CNN VGG16

📄 Page 22 — Architecture VGG16
Ce qu'on voit :
Schéma du réseau :
- Conv2D
- MaxPooling
- Dense

Vecteur final = 4096 dimensions
À dire :

Le CNN extrait des représentations de haut niveau, plus abstraites que SIFT.

Page 23 — T-SNE CNN
Ce qu'on voit :
Séparation beaucoup plus nette des groupes.
ARI ≈ 0.45
Message clé :

Les images se regroupent naturellement par catégorie.
On voit des clusters bien formés.

Page 24 — Matrice de confusion CNN
Ce qu'on voit :
Beaucoup plus de valeurs sur la diagonale.
Interprétation :

Les clusters correspondent largement aux vraies classes.

PARTIE CLASSIFICATION SUPERVISÉE

Page 27 — Approche simple
Graphique Accuracy :
Courbe train monte vers ~0.98
Validation stagne vers ~0.85
Écart visible → overfitting
Graphique Loss :
Train loss → presque 0
Val loss → stable autour de 1
Explication :

Le modèle apprend très bien les données d'entraînement, mais généralise moins bien.
Test accuracy = 0.7952

Page 28 — Matrice confusion approche simple
Ce qu'on voit :
Majorité sur diagonale.
Quelques confusions :
- Home Decor vs Home Furnishing

À dire :

Les catégories visuellement proches sont parfois confondues.

Page 29 — ImageDataGenerator
Graphiques :
Moins d'écart train/val
Courbes plus proches
Accuracy test = 0.7619
Interprétation :

L'augmentation rend l'apprentissage plus difficile mais réduit le surapprentissage.

Page 30 — Dataset sans augmentation
Courbes similaires à baseline
Accuracy test = 0.7905
Pipeline plus moderne.

Page 31 — Dataset + augmentation intégrée
Graphique Accuracy :
Train ≈ 0.91
Test ≈ 0.84
Graphique Loss :
Test loss ≈ 0.80
Graphique Learning rate :
Réduction automatique (ReduceLROnPlateau)
Message clé :

Meilleur compromis entre performance et généralisation.
Accuracy test = 0.8381

Résumé visuel global

Méthode	ARI / Accuracy	Lecture visuelle
Texte simple	faible	clusters mélangés
BERT	bon	clusters nets
SIFT	très faible	fort mélange
CNN	bon	séparation claire
Supervised final	83.8%	meilleure solution

Qu’est-ce qu’une couche ?
Un réseau de neurones est composé de **plusieurs couches empilées**.
Chaque couche :
- reçoit des données en entrée
- applique une transformation mathématique
- transmet le résultat à la couche suivante

C’est comme une chaîne d’analyse progressive.

Les couches dans le traitement d’image
Dans ton projet, tu utilises **VGG16**, qui contient plusieurs types de couches.

◆ A. Conv2D (Convolution)
Ce sont les couches les plus importantes pour les images.
Rôle :
Elles détectent des motifs visuels :
- Bords
- Textures
- Formes simples
- Puis formes complexes
- Puis objets

Exemple :
- 1ère couche → détecte des lignes
- 5ème couche → détecte des formes
- Dernières couches → détectent des objets complets

Dans ton slide page 22, on voit ces blocs Conv2D.

◆ B. MaxPooling
Sert à réduire la taille de l’image.
Rôle :
- Diminue le nombre de pixels
- Garde l’information importante
- Réduit le calcul

C’est une compression intelligente.

◆ C. Dense (Fully Connected)
Ce sont les couches finales.
Rôle :
- Faire du “raisonnement”
- Prendre la décision finale

Dans ton projet, tu as ajouté :
Dense(256, activation="relu")
Dense(7, activation="softmax")

◆ D. Dropout
Sert à éviter le surapprentissage.
Pendant l’entraînement :
- 50% des neurones sont désactivés aléatoirement
- Cela force le modèle à généraliser

3 Structure simplifiée de VGG16
Image → Conv → Conv → Pool
→ Conv → Conv → Pool
→ Conv → Conv → Pool
→ Dense → Dense
À la fin :
➡ Chaque image devient un vecteur de 4096 nombres
C’est ce que tu utilises pour le clustering et la classification.

4 Les couches dans TON projet (Transfer Learning)

Dans ton projet :
Tu as fait :

```
for layer in base_model.layers:
    layer.trainable = False
```

Cela signifie :
Tu “gèles” les couches convolutionnelles.
Pourquoi ?
Parce qu’elles sont déjà entraînées sur ImageNet (1,4 million d’images).
Elles savent déjà détecter des motifs généraux.
Ensuite tu ajoutes :

- Tes propres couches Dense
- Spécifiques à tes 7 catégories

C’est ça le **Transfer Learning**.

5 Pourquoi c’est puissant ?

- Tu n’as que 1050 images
- Entraîner un CNN from scratch serait trop risqué
- VGG16 apporte une connaissance visuelle pré-apprise

Résultat :
👉 ARI ≈ 0.45 en clustering
👉 Accuracy ≈ 83.8% en supervisé

6 Différence avec SIFT

SIFT :

- détecte des textures locales
- ne comprend pas les objets

CNN :

- apprend des représentations hiérarchiques
- comprend des structures globales

C’est pour ça que ton ARI passe de 0.08 à 0.45.

Un CNN est composé de couches successives.
Les premières couches détectent des **formes simples** comme des lignes.
Les couches intermédiaires détectent **des motifs plus complexes**.
Les dernières couches détectent des **objets complets**.
Dans mon projet, j’ai utilisé VGG16 en Transfer Learning : j’ai conservé ses couches convolutionnelles pré-entraînées et ajouté mes propres couches finales pour adapter le modèle à mes 7 catégories.

❓ Pourquoi avoir gelé les couches ? :

Pour éviter le surapprentissage et profiter des représentations déjà apprises sur ImageNet.

❓ Qu’est-ce qu’une couche convolutionnelle ?

C’est une couche qui applique des filtres sur l’image pour détecter des motifs locaux comme des bords, textures ou formes.

1 Données & Analyse

- **Dataset**

Ensemble de données utilisées pour entraîner un modèle.
→ Dans ton projet : images + descriptions produits.

- **EDA (Exploratory Data Analysis)**

Analyse exploratoire des données.
→ Permet de comprendre la structure, la qualité et les déséquilibres.

- **Preprocessing**

Nettoyage et préparation des données avant modélisation.
→ Suppression stopwords, redimensionnement images, normalisation.

- **Feature**

Information numérique extraite des données.
→ Un mot transformé en nombre, ou une image transformée en vecteur.

- **Feature Extraction**

Processus de transformation des données brutes en vecteurs numériques exploitables.

2 NLP (Texte)

- **Bag of Words**

Représentation texte basée sur le comptage des mots.

- **TF-IDF**

Mesure l'importance d'un mot dans un document.

- **Word2Vec**

Méthode qui transforme les mots en vecteurs en capturant leur proximité sémantique.

- **Embedding**

Représentation numérique dense d'un mot ou d'une phrase.

- **BERT**

Modèle avancé qui comprend le contexte d'un mot dans une phrase.

- **Clustering**

Regroupement automatique de données similaires sans connaître les labels.

3 Vision par Ordinateur

- **CNN (Convolutional Neural Network)**

Réseau de neurones spécialisé dans le traitement d'images.

- **Convolution**

Opération qui applique un filtre pour détecter des motifs visuels.

- **Filtre (Kernel)**

Petite matrice qui détecte des motifs (bords, textures).

- **Pooling**

Réduction de la taille des données tout en gardant l'information importante.

- **SIFT**

Méthode classique de détection de points clés dans une image.

- **BoVW (Bag of Visual Words)**

Représentation d'image basée sur des histogrammes de motifs visuels.

- **VGG16**

Réseau CNN pré-entraîné utilisé pour extraire des features images.

- **Transfer Learning**

Réutilisation d'un modèle pré-entraîné pour un nouveau problème.

📉 4 Réduction & Clustering

- **T-SNE**

Algorithme de projection non linéaire pour visualiser en 2D.

- **KMeans**

Algorithme de clustering qui regroupe les points en k groupes.

- **Silhouette Score**

Mesure la qualité d'un clustering (cohésion vs séparation).

5 Classification Supervisée

◆ **Classification supervisée**
Apprentissage avec des données étiquetées.

◆ **Label**
Catégorie réelle d'un produit.

◆ **Train set**
Données utilisées pour entraîner le modèle.

◆ **Validation set**
Données utilisées pour ajuster le modèle.

◆ **Test set**
Données utilisées pour évaluer les performances finales.

◆ **Pipeline**
Enchaînement structuré des étapes du traitement.

🏁 **3 mots ultra importants à maîtriser**
Si tu devais en retenir 3 pour la soutenance :

1. **Transfer Learning**
2. **Feature Extraction**
3. **Généralisation**

6 Métriques

◆ **Accuracy**
Pourcentage de prédictions correctes.

◆ **Loss**
Mesure de l'erreur du modèle.

◆ **Overfitting**
Le modèle apprend trop bien les données d'entraînement et généralise mal.

◆ **Underfitting**
Le modèle n'apprend pas suffisamment.

◆ **Matrice de confusion**
Tableau montrant les erreurs de classification par classe.

7 Entraînement

◆ **Epoch**
Un passage complet sur toutes les données d'entraînement.

◆ **Batch**
Petit groupe de données traité en une fois.

◆ **Learning Rate**
Vitesse à laquelle le modèle ajuste ses poids.

◆ **Early Stopping**
Arrêt automatique si la performance ne s'améliore plus.

◆ **Dropout**
Technique pour éviter le surapprentissage en désactivant des neurones.

◆ **Data Augmentation**
Transformation d'images (rotation, flip…) pour enrichir les données.

8 Pipeline TensorFlow

◆ **ImageDataGenerator**
Ancienne méthode pour charger et augmenter les images.

◆ **image_dataset_from_directory**
Méthode moderne TensorFlow pour charger les images efficacement.

◆ **Dataset API**
Pipeline optimisé pour l'entraînement GPU.

9 Concepts clés globaux

◆ **Faisabilité**
Vérification que les données peuvent être séparées naturellement.

◆ **Généralisation**
Capacité du modèle à bien fonctionner sur des données nouvelles.